



Universidad Nacional Autónoma de México



Facultad de Ingeniería

Integrantes:

Espinoza Matamoros Percival Ulises - 320025561

Flores Colin Victor Jaziel - 320266083

Lara Hernandez Angel Husiel - 320060829

Laboratorio de Microcomputadoras

Grupo: 06 - Semestre: 2026-2

Cuestionario Previo 2:

Programación en ensamblador; direccionamiento indirecto

Profesor:

Ing. Moises Melendez Reyes

Fecha de Entrega:

22 de Febrero del 2026



1.-Explique que son los modos de direccionamiento en la programación de microprocesadores.

Los modos de direccionamiento en los microprocesadores es el metodo en el que mediante el cual se calcula la direccion efectiva de los operadores, no obstante para cualquier programador de ensablandor es esencial comprender estos modos de direccionamiento, ya que el direccionamiento no solo define donde esta el dato, sino que tambien cuanto tiempo y energia costara el acceso al mismo.

Los modos de direccionamiento han ido evolucionando mas entre las arquitecturas CISC y RISC, mientras que las arquitecturas CISC antiguas permitían operaciones aritméticas directamente sobre direcciones de memoria complejas.

Para esto podemos tener las clasificaciones y sus modos basandose en el ubicacion del operador:

1. Inmediato: El dato es parte de la instrucción.
2. Registro: El dato está en el banco de registros.
3. Directo/Absoluto: La dirección de memoria es constante.
4. Indirecto/Indexado: La dirección se calcula dinámicamente.

No obstante, esta clasificacion no es estatica, tenemos el ejemplos de la Raspberry Pi, que es una arquitectura RISC, y a pesar de esto tiene modos de direccionamiento complejos como el modo de direccionamiento indirecto, que es un modo de direccionamiento que se caracteriza por utilizar un registro para almacenar la dirección de memoria del operando, en lugar de incluir la dirección directamente en la instrucción.

Ahora bien, tenemos el caso de la direccion efectiva, donde es el resultado final de cualquier calculo de direccion requerido por una instrucción antes de acceder a la memoria. La eficiencia con la que una microarquitectura puede calcular esta direccion efectiva determina el rendimiento en tareas críticas como el recorrido de arreglos, la manipulación de pilas y el acceso a estructuras de datos complejas.

La implementación hardware de estos modos implica el uso de multiplexores para seleccionar entre diferentes fuentes de datos (el contador de programa, un registro, o un campo de la instrucción) y sumadores para calcular la dirección final.



2.-Describa el modo de direccionamiento inherente , escriba un ejemplo y un gráfico que represente este modo de direccionamiento.

El modo de direccionamiento inherente se conoce tambien como direccionamiento implicito, donde este lo que hace es una representacion mas primitiva y eficiente de instrucciones, en este modo no se especifica explicitamente mediante campos de direccion o seleccion de registros, de hecho es lo opuesto, donde la ubicacion del operando es fija y se asume que el operando esta en un lugar predefinido, como por ejemplo el acumulador o un registro especifico.

No obstante la instrrcciones que modifican el registro de estado, instrucciones de barrera de memoria o tambien las intriccion de NO OPERACION caen en esta categoria conceptual de implicito. Dicho en otras palabras lo anteriormente explicado en el parrafo anterior, lo que hace escencialmente es que el procesador ya entiende implicitamente que no debe alterar ningun estado de los registros, o que debe modificar el estado de los flags, o que no se va a realizar ninguna operacion, lo que hace que este modo de direccionamiento sea muy eficiente en terminos de ciclos de reloj y consumo de energia.

Ejemplo:

En el caso de computadoras modernas tenemos el caso de las intrucciones NOP (No Operation), que son instrucciones que no realizan ninguna operación pero que pueden ser útiles para sincronización o para rellenar espacio en el código. En este caso, la instrucción NOP es un ejemplo clásico de direccionamiento inherente, ya que no requiere operandos y su efecto es simplemente no hacer nada.

Codigo ensamblador:

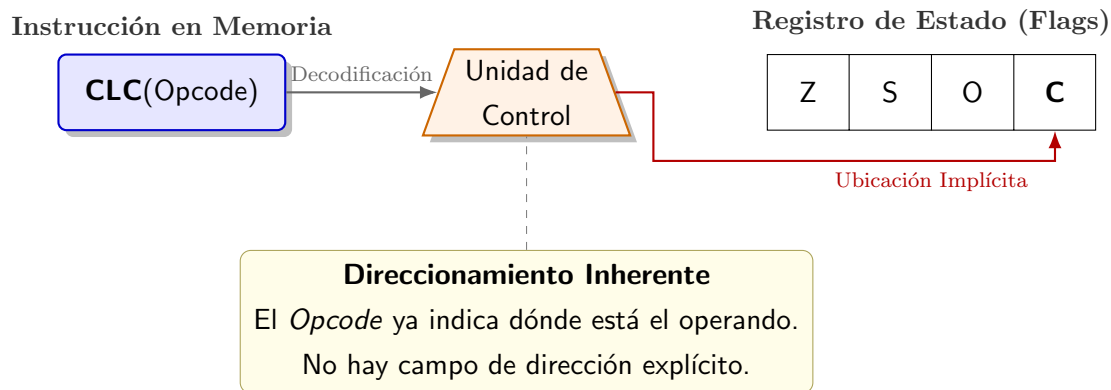
Listing 1: Ejemplo de direccionamiento inherente con NOP

```
1  NOP ; No Operation, no hace nada
```

Al decodificar este opcode, la Unidad de Control sabe implícitamente que no debe activar ninguna línea de lectura de memoria ni habilitar la escritura en ningún registro. El .ºoperando.^{es} el propio flujo de tiempo del procesador, consumiendo un ciclo sin efectos secundarios.

Otro ejemplo en microprocesadores, es la instrucción CLC (Clear Carry). El operando es el bit de acarreo (Carry Flag) en el registro de estado, la instrucción CLC lleva implícita la dirección de ese bit específico..

Diagrama:



En la parte izquierda, se observa que la instrucción reside en memoria conteniendo únicamente el Opcode, sin campos adicionales para operandos. Al pasar a la Unidad de Control, esta decodifica la instrucción y sabe implícitamente que debe actuar sobre un recurso específico del procesador: el Flag de Acarreo (C) dentro del Registro de Estado. Como muestra la línea roja, la ubicación del operando destino no se obtiene de una dirección de memoria ni de un índice de registro, lo que resulta en una ejecución rápida y directa.

3.-Describa el modo de direccionamiento inmediato, escriba un ejemplo y un gráfico que represente este modo de direccionamiento.

En este modo de direccionamiento inmediato, el operador fuente no se queda en una dirección de memoria ni en otro registro, sino que lo que hace es que está codificado directamente dentro de los bits que conforman la instrucción en sí, en otras palabras es que el dato es inmediatamente accesible después de la búsqueda de la instrucción.

La implementación de este modo tanto en arquitectura RISC de 32 bits o ARM es difícil de implementar en ambos casos, donde en sí el problema principal es en la geometría de la instrucción, por ejemplo si una instrucción tiene una longitud total de 32 bits y debe contener el código de operación, los códigos de condición y la dirección del registro de destino, es técnicamente imposible reservar 32 bits adicionales dentro de ella para cargar una constante arbitraria de 32 bits. Para este problema, se presenta una solución, donde en arquitecturas ARM, se hace uso del esquema de rotación, donde en lugar de almacenar la constante tal cual, la instrucción almacena la constante tal cual, la instrucción almacena un valor base de 8 bits y un valor de rotación de 4 bits.

No obstante si se intenta usar una constante que no puede generarse mediante esta rotación

(por ejemplo, una constante arbitraria de 32 bits con muchos cambios de bit), el ensamblador generará un error o, si es inteligente, convertirá la instrucción en una carga de memoria (LDR) desde un "literal pool", cambiando efectivamente el modo de direccionamiento de inmediato a relativo al PC, un matiz crítico que Smith advierte a los programadores.

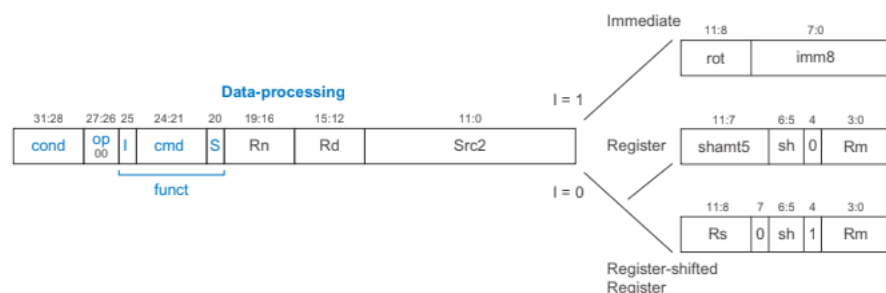
Ejemplo:

En el caso de queremos iniciar un contador en 10, entonces la instrucción en ensamblador sería:

```
MOV R0, #10
```

Aquí, el símbolo # (o a veces \$) denota que el 10 es un valor inmediato. El procesador toma el campo de la instrucción correspondiente al operando 2, lo pasa por el "barrel shifter" (desplazador de barril) descrito por Elahi, y deposita el resultado en R0. No se genera ninguna transacción en el bus de memoria de datos.

Diagrama:



Para esto se muestra cómo el operando (dato constante) se encuentra codificado directamente dentro de la propia instrucción, ocupando campos específicos (valor base y rotación) en lugar de residir en una dirección de memoria externa. Este valor es extraído y procesado por el Barrel Shifter para generar la constante final de 32 bits, la cual se deposita inmediatamente en el registro de destino (R0), evitando así cualquier ciclo adicional de lectura en el bus de datos.



4.-Describa el modo de direccionamiento directo, escriba un ejemplo y un gráfico que represente este modo de direccionamiento.

El modo de direccionamiento directo también es conocido como direccionamiento absoluto, este define el método de acceso a memoria en el cual la instrucción contiene, dentro de sus bits la dirección efectiva que también es conocida como EA, este como tal no se requiere ningún cálculo complejo, sino que lo que hace la unidad es que el control del procesador extrae el valor del campo de dirección de la instrucción y lo coloca directamente en el bus de direcciones del sistema para iniciar un ciclo de lectura y escritura.

Para esto el modo de direccionamiento directo es muy rápido, donde este ya no utiliza la unidad lógica o mejor conocida como ALU para calcular la dirección, no obstante, este modelo sufre de algo conocido como la rigidez, donde si el programa se mueve a otra dirección de memoria entonces las instrucciones tienen que ser reubicadas esto por el carácter del sistema operativo.

Cabe recalcar que la implementación de este modo de direccionamiento varía, donde tenemos los siguientes:

1. Direccionamiento de página cero: En este se utiliza una dirección abreviada para acceder rápidamente a los primeros 256 bytes de la memoria.
2. Direccionamiento extendido: Utiliza la dirección completa de memoria, pero en arquitectura RISC esto a menudo requiere más ciclos debido a las limitaciones del ancho de palabra mencionadas anteriormente.

Cabe recalcar un caso importante, donde si una instrucción LDR reservara 12 bits para un desplazamiento o dirección inmediata, solo podría direccionar un rango de 4096 bytes. Esto hace que el "direccionamiento directo puro" que es como la inclusión de una dirección completa de 32 bits dentro de una sola instrucción sea técnicamente imposible en una sola instrucción de máquina estándar de 32 bits.

Esto hace que el direccionamiento directo en lenguajes de alto nivel o en ensamblador simplificado, se implementa en la microarquitectura ARM mediante técnicas diferentes como el direccionamiento relativo al PC o el uso de pares de instrucciones (MOV + MOVt) para construir la dirección completa en un registro antes de acceder a la memoria.

Ejemplo:

Para este ejemplo tendremos el caso de un sistema basado en la arquitectura ARM, donde necesitamos recuperar un valor de configuración almacenado en una dirección específica de memoria, la dirección será la 0x2050 y cargarlo en el registro R0 para ser procesado.

La instrucción en ensamblador sería:

Listing 2: Ejemplo de direccionamiento directo

```
1 LDR R0, [0x2050] ; Cargar en R0 el valor contenido en la
    direccion 0x2050
```

Ahora para esto tendremos que el Fetch, carga la instrucción desde la memoria de programa, después el Decode, interpreta el opcode LDR y reconoce que se trata de una instrucción de carga con direccionamiento directo. El control del procesador extrae el valor 0x2050 del campo de dirección, en la implementación de 32 bits entonces si esta dirección no cabe en el campo inmediato, el ensamblador habrá colocado la dirección en una “literal pool” cercana, y la instrucción real será:

Listing 3: Ejemplo de direccionamiento directo con literal pool

```
1 LDR R0, [PC, #offset] ; Cargar en R0 el valor contenido en
    la direccion apuntada por PC + offset
```

Para leer esta dirección primero, o usará instrucciones MOV/MOVT para contruir la dirección en un registro temporal, después la lectura de memoria de la CPU coloca la dirección en el bus de dirección y se activa la señal de control de lectura, después el la transferencia de datos de la memoria responde colocando el dato almacenado en la celda “0x2050” en el bus de datos, y finalmente el procesador carga este valor en R0 para su uso posterior.

Con esto tuvimos el flujo directo: Instrucción → Dirección → Dato.

Diagrama:

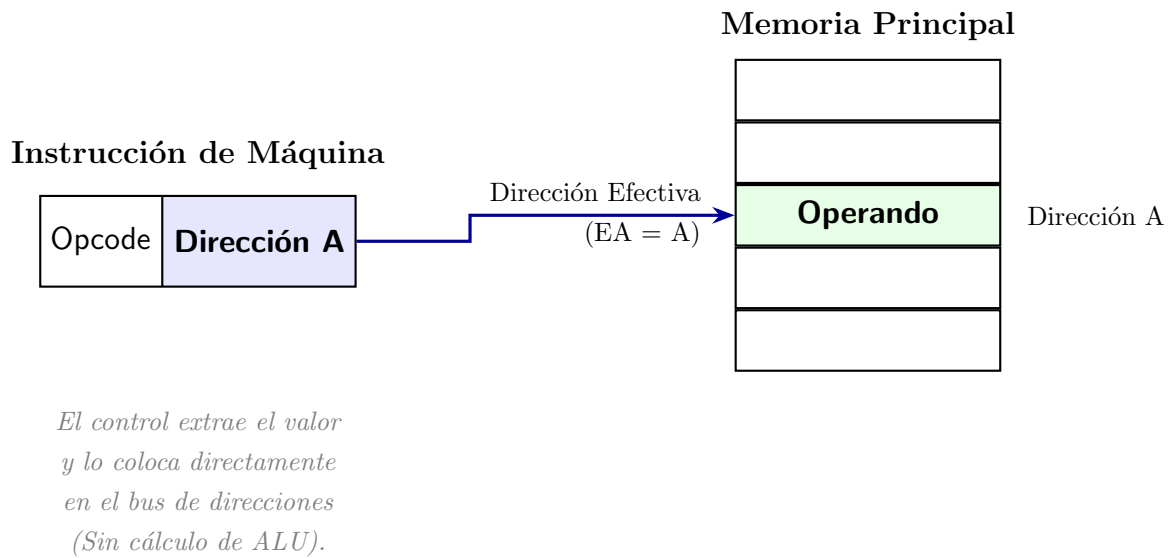


Figura 1: Esquema del Modo de Direccionamiento Directo.

5.-Describa el modo de direccionamiento indirecto, escriba un ejemplo y un gráfico que represente este modo de direccionamiento.

El direccionamiento indirecto es la forma en que se hace uso la computacion dinamica, donde se implementan punteros, arreglos, estructuras de datos complejas y el paso de parametros por referencias.

No obstante, para la arquitectura RISC y ARM el direccionamiento indirecto es la forma en que se accede a la memoria de datos, donde debido a las instrucciones no pueden tener direcciones completas, la arquitectura usa los registros de proposito general como ounteros o direcciones base.

En otras palabras En el direccionamiento indirecto, la instrucción no especifica donde esta el dato, donde en este caso este se guarda en el registro base R_n y la direccion efectiva se calcula viendo el contenido del registro base.

Para esto tenemos variaciones de esta estructura o forma de direccionamiento:

1. Direccionamiento indirecto por registro: Este define como pre-indexado sin desplazamiento, esta es que la direccion efectiva simplemente el contenido del registro R_n es equivalente a un puntero simple (*ptr)-
2. Direccionamiento Pre-indexado con Desplazamiento: La dirección efectiva se calcula

sumando un desplazamiento inmediato al registro base ($EA = R_n + \text{offset}$). El registro base R_n no se modifica permanentemente, solo se usa para el cálculo. Esto es útil para acceder a campos dentro de una estructura.

3. Direccionamiento Pre-indexado con Desplazamiento de Registro: El desplazamiento no es una constante, sino el valor contenido en otro registro R_m . Esto se utiliza para acceder a arreglos donde el índice es variable.
4. Direccionamiento Pre-indexado con Registro Escalado: En la arquitectura ARM el registro de índice R_m pasa por el desplazador de barril antes de sumarse a la base. Esto permite calcular direcciones de arreglos de palabras (4 bytes) en un solo ciclo, escalando el índice por 4 automáticamente.
5. Modos con Actualización (Write-back): En la parte de pre-indexado tenemos que calcula la dirección, accede a la memoria y luego actualiza el registro base con la nueva dirección. En la parte de Post-indexado tenemos que accede a la memoria usando la dirección base original y luego actualiza el registro base sumando el desplazamiento.

En una instrucción de carga (LDR) con direccionamiento indirecto:

1. Etapa de Decodificación: La unidad de control identifica los registros base (R_n) y destino (R_d).
2. Lectura de Registros: Se lee el valor de R_n del banco de registros. Si hay un desplazamiento de registro (R_m), también se lee.
3. Etapa de Ejecución (ALU): La ALU no realiza operaciones aritméticas para el usuario, sino que calcula la dirección. Suma la base (R_n) con el desplazamiento.
4. Etapa de Memoria: La Dirección Efectiva se presenta al bus de direcciones de la memoria de datos para leer el valor.
5. Etapa de Escritura: El dato leído de la memoria se escribe en el registro destino (R_d).

Ejemplo:

Supongamos que queremos acceder a un elemento de un arreglo de enteros almacenado en memoria, donde el puntero al inicio del arreglo está en $R1$ y el índice del elemento que queremos acceder está en $R2$. La instrucción en ensamblador ARM podría ser:

Listing 4: Ejemplo de direccionamiento indirecto con desplazamiento de registro

```
1      LDR R0, [R1, R2, LSL #2] ; Cargar en R0 el valor del
      arreglo en la posición R2
```

En este ejemplo, $R1$ contiene la dirección base del arreglo, $R2$ contiene el índice del elemento que queremos acceder, y el desplazamiento se calcula multiplicando el índice por 4 (tamaño de un entero) usando el desplazador de barril (LSL #2). La ALU calcula la dirección efectiva sumando $R1$ con el resultado del desplazamiento, y luego se accede a la memoria para cargar el valor en $R0$.

Listing 5: Movimiento de memoria en direccionamiento indirecto con desplazamiento

```
1      ; 1. INICIALIZACION (Estado previo)
2      MOV R1, #0x1000      ; R1 contiene la dirección base del arreglo
3      MOV R2, #2           ; R2 contiene el índice (queremos acceder al
      elemento 2)
4
5      ; MAPA DE MEMORIA ACTUAL:
6      ; -----
7      ; Dirección | Contenido (Valores del Arreglo) | Desc
8      ; 0x1000    | 10                               | Elemento 0
9      ; 0x1004    | 25                               | Elemento 1
10     ; 0x1008    | 42                               | Elemento 2 <--
      Objetivo
11     ; 0x100C    | 55                               | Elemento 3
12
13     ; 2. EJECUCION DE LA INSTRUCCION DE CARGA INDIRECTA
14     LDR R0, [R1, R2, LSL #2]
15
16     ; EXPLICACION DEL FLUJO DE DATOS (Hardware):
17     ; Paso A: El desplazador de barril toma R2 (2) y hace un "Logical
      Shift Left"
18     ;           de 2 bits (LSL #2), lo que equivale a multiplicar por 4.
      Resultado = 8.
```

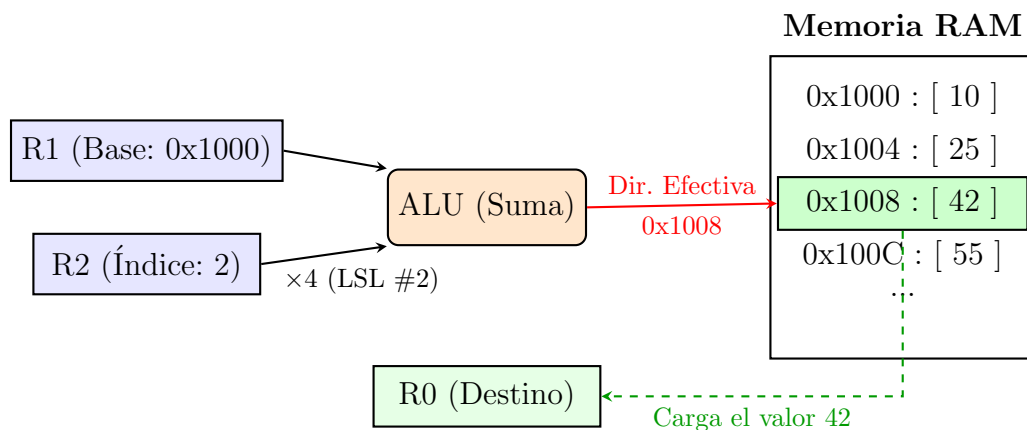
```

19      ; Paso B: La ALU suma el desplazamiento a la direccion base en R1.
20      ;           Dir. Efectiva = 0x1000 + 8 = 0x1008.
21      ; Paso C: El bus de direcciones apunta a 0x1008 en la memoria RAM.
22      ; Paso D: El bus de datos lee el contenido en esa direccion (42) y
                lo envia a R0.
23
24      ; 3. ESTADO FINAL
25      ; R0 = 42 (El registro R0 ahora contiene el valor extraido de la
                memoria)

```

Como se observa en el flujo, la instrucción de lectura nunca supo que iba a buscar información a la dirección '0x1008' al momento de ser escrita; esta dirección se calculó dinámicamente en tiempo de ejecución combinando los registros y el desplazamiento, demostrando el poder y la flexibilidad del direccionamiento indirecto.

Diagrama:



En el modo de direccionamiento indirecto, la Unidad Lógico Aritmética (ALU) calcula la dirección efectiva de memoria sumando el valor del registro base R1 (0x1000) con el registro índice R2 (2). Este último se multiplica por 4 mediante un desplazamiento lógico a la izquierda (LSL #2) para obtener un desplazamiento real de 8. Al sumar ambos valores, la ALU genera la dirección efectiva final (0x1000 + 8 = 0x1008), apuntando a una celda específica en la Memoria RAM de la cual se extrae el dato contenido (42) para ser cargado finalmente en el registro de destino R0.



Referencias

- Anaya, R. (s.f.). *Manual de recursos y aplicaciones Plataforma Raspberry Pi*. <https://odin.fib.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf>
- Elahi, A. (2022, 17 de marzo). *Computer systems: Digital Design, Fundamentals of Computer Architecture and ARM Assembly Language* (2.^a ed.). Springer. <https://doi.org/10.1007/978-3-030-93449-1>
- Harris, D., & Harris, S. (2015, 22 de abril). *Digital Design and Computer Architecture* (Arm Edition). Morgan Kaufmann Pub.
- Smith, S. (2019, octubre). *Raspberry Pi Assembly Language Programming*. Apress. <https://doi.org/10.1007/978-1-4842-5287-1>